

Learning Objectives

After completing this project, you should be able to:

- Organize real-world appointment data using lists and dictionaries.
- Validate information before storing records.
- Detect scheduling conflicts.
- Search, update, and manage records.
- Work with dates and schedules.
- Use `st.session_state` to maintain application data.
- Build a multi-tab Streamlit application with forms, sidebar, columns, and status elements.
- Apply competition-style UI requirements including **colors, alignment, buttons, icons, borders, and dividers**.
- Generate useful clinic statistics from stored appointment data.

Problem 16 – Smart Warehouse Inventory Manager

Difficulty: ★ ★ ★ ★ Intermediate

Platform: Streamlit only

No console interface

Overview

Build a Streamlit application for managing the inventory of a small warehouse.

The application should allow warehouse staff to:

- Add products
- View inventory
- Search for products
- Restock products
- Remove stock
- Delete products
- Update prices
- Identify low-stock products
- View inventory statistics

The application must validate user input and prevent duplicate Product IDs and negative stock quantities.

Your application should have a clean and organized Streamlit interface using **tabs, columns, a sidebar, forms, buttons, status messages, text formatting, and basic page configuration**.

Part 1 — Store Inventory Data

Create a suitable Python data structure for storing the inventory.

Each product must contain:

- Product ID
- Product name
- Category
- Quantity
- Unit price

You may use a **list of dictionaries** or another suitable structure.

Example:

```
inventory = [ { "id": "P101", "name": "Wireless Mouse", "category": "Electronics",  
"quantity": 25, "price": 15.99 } ]
```

You may design your own structure.

Part 2 — Application Setup

Configure the Streamlit page using:

- `st.set_page_config()`
- `st.title()`
- `st.header()`
- `st.subheader()`

The page should have:

- A suitable page title
- A suitable page icon
- A clear application title

Formatting requirement

Use **text alignment** where appropriate.

For example:

```
st.title("Smart Warehouse Inventory Manager", text_alignment="center")
```

You may choose the alignment that best fits each section.

Part 3 — Sidebar

Create a sidebar containing:

Warehouse Information

Display:

- Warehouse name
- Total number of products
- Total quantity in stock
- Total inventory value

Navigation

Use a suitable Streamlit widget to allow the user to move between the main sections of the application.

The navigation should be clearly separated from the inventory summary.

Part 4 — Add Product

Create an **Add Product** section using:

`st.form()`

The form must contain:

- Product ID → `st.text_input()`
- Product name → `st.text_input()`
- Category → `st.selectbox()`
- Quantity → `st.number_input()`
- Unit price → `st.number_input()`
- Add Product → form submit button

Validation

Before adding the product:

1. Product ID must not be empty.
2. Product name must not be empty.
3. Quantity must be greater than or equal to 0.
4. Unit price must be greater than 0.
5. Product ID must not already exist.

If the product is valid:

Product added successfully.

If the Product ID already exists, display an error.

Part 5 — View Inventory

Create an **Inventory** section.

Display all products in an organized table.

The table should contain:

Product ID Name Category Quantity Unit Price

You may use:

`st.write()`

or another appropriate Streamlit display method.

If there are no products, display an informational message.

Part 6 — Search Products

Create a search section.

The user should be able to select a search method:

- Product ID
- Product Name
- Category

Then enter the search text.

Display all matching products.

Rules

- Product ID searches should match the ID.
- Product name searches should be **case-insensitive**.
- Category searches should be **case-insensitive**.
- If no products match, display an informational message.

Example:

No matching products were found.

Part 7 — Stock Management

Create a **Stock Management** section.

The user must first select an existing product.

Provide two separate operations:

A. Restock Product

Allow the user to enter the quantity to add.

Rules:

- Quantity must be greater than 0.
- Add the quantity to the existing stock.
- Display the new stock quantity.

Example:

Stock updated successfully.

Current Stock: 40

B. Remove Stock

Allow the user to enter the quantity leaving the warehouse.

Rules:

- Quantity must be greater than 0.
- The requested quantity cannot be greater than the available stock.
- Stock must never become negative.

Example:

Stock removed successfully.

Remaining Stock: 18

Part 8 — Delete Product

Allow the user to select an existing product.

Display the selected product's information before deletion.

Require confirmation before permanently deleting it.

For example:

Are you sure you want to delete P101?

Only delete the product after confirmation.

After successful deletion:

Product deleted successfully.

Part 9 — Update Product Price ★

Add a product-price update feature.

The user should:

1. Select an existing product.
2. Enter a new unit price.
3. Confirm the update.

Only the **unit price** should change.

The product's:

- ID
- Name
- Category
- Quantity

must remain unchanged.

Display a success message after updating.

Part 10 — Low-Stock Report

Allow the user to choose a low-stock threshold.

For example:

Show products with stock below:

[5]

Display every product whose quantity is **less than the selected threshold**.

Example:

Low Stock Products

USB Cable 3

Notebook 2

Ink Cartridge 1

If there are no low-stock products:

All products currently have sufficient stock.

Use a warning message when low-stock products exist.

Part 11 — Inventory Statistics

Create an **Inventory Statistics** section.

Calculate and display:

1. Total Products

Number of products stored.

2. Total Quantity

Total number of items across all products.

3. Total Inventory Value

Calculate:

quantity × unit price

for every product and add the results.

4. Highest Stock Product

Display the product with the largest quantity.

5. Lowest Stock Product

Display the product with the smallest quantity.

6. Products Per Category

Calculate how many products belong to each category.

Example:

Electronics: 8

Office Supplies: 9

Furniture: 7

Part 12 — Streamlit Status Messages

Your application must use **all five** of the following:

`st.success()``st.info()``st.warning()``st.error()``st.spinner()`

Use them appropriately.

st.success()

Use after successful operations:

- Product added
- Stock restocked
- Stock removed
- Product deleted
- Price updated

st.info()

Use for informational situations:

- Inventory is empty
- No search results
- No low-stock products

st.warning()

Use when:

- Products are low on stock
- The user is about to perform a potentially destructive operation

st.error()

Use for invalid operations:

- Duplicate Product ID
- Empty required field
- Invalid quantity
- Insufficient stock
- Invalid price

st.spinner()

Use while processing an inventory operation.

For example:

with `st.spinner("Updating inventory...")`: ...

Part 13 — Layout Requirements

Your application must use:

Sidebar

Use:

st.sidebar

for:

- Warehouse name
 - Navigation
 - Inventory summary
-

Tabs

Create **four main tabs**:

Add Product | Inventory | Stock Management | Reports

Tab 1 — Add Product

Contains:

- Product form
- Validation
- Add Product button

Tab 2 — Inventory

Contains:

- Complete inventory
- Search products

Tab 3 — Stock Management

Contains:

- Restock
- Remove stock
- Update price
- Delete product

Tab 4 — Reports

Contains:

- Low-stock report
 - Inventory statistics
-

Part 14 — Columns

Use `st.columns()` to organize information.

For example, inside the Reports tab:

LOW STOCK REPORT	INVENTORY STATISTICS	

You should decide where columns improve the interface rather than creating columns unnecessarily.

Part 15 — Text Formatting 🧐

Your application must demonstrate **three types of visual formatting**.

1. Text Colors

Use Streamlit's supported text/color formatting where appropriate.

For example:

```
st.markdown("***Low Stock Products** :red[🚨]")
```

Use colors to highlight important information such as:

- Low stock
- Success
- Important totals

Do not color every piece of text.

2. Primary and Secondary Buttons

Use both button types:

```
st.button("Add Product", type="primary")
```

and

```
st.button("Cancel", type="secondary")
```

Use **primary buttons** for important actions.

Use **secondary buttons** for less important or cancellation actions.

3. Text Alignment

Use text alignment for appropriate headings or important information.

For example:

```
st.title( "Smart Warehouse Inventory Manager", text_alignment="center")
```

4. New Formatting Option — Dividers

Use:

```
st.divider()
```

to visually separate major sections.

For example:

Inventory Statistics

Stock Information

The goal is to make the application easier to read without over-formatting it.

Part 16 — Required Streamlit Functions

Your project must demonstrate all of these:

Text / Display

- `st.set_page_config()`
- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`

Input

- `st.text_input()`
- `st.number_input()`
- `st.selectbox()`
- `st.button()`

Layout

- `st.form()`
- `st.columns()`
- `st.sidebar`
- `st.tabs()`

Status

- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Part 17 — Important Streamlit Rule

This is a **Streamlit-only project**.

Do **not** use:

`input()``print()``while True`

Do not create a console menu such as:

1. Add Product
2. View Inventory
3. Search

The **Streamlit interface** replaces the console menu.

Use:

- Tabs
- Sidebar
- Buttons
- Forms
- Selectboxes
- Number inputs
- Text inputs

to control the application.

★ Bonus Mission

Add an **Inventory Value by Category** report.

For example:

Electronics \$4,250.50

Office Supplies \$2,180.00

Furniture \$6,130.25

This requires you to combine:

- Dictionaries
 - Loops
 - Conditions
 - Arithmetic
 - Searching/grouping
-

★★★ Challenge Mission — Multi-Warehouse

Upgrade the application so it can manage **multiple warehouses**.

Each warehouse should have its own inventory.

Add the ability to:

- Create multiple warehouses
- Select a warehouse
- Add products to a specific warehouse
- View warehouse inventory
- Transfer stock between warehouses
- Prevent transfers greater than available stock
- Display total stock per warehouse
- Display inventory value per warehouse
- Generate category summaries
- Record stock additions and removals

Advanced Challenge

Create an activity report such as:

Today's Warehouse Activity

P101

+20 Restocked

P205

-5 Removed

P310

+15 Restocked

This challenge is optional and should only be attempted after completing the main project.

Python Topics Covered

This version focuses on the topics actually needed for the project:

- Variables
- Comments
- Data types
- Strings
- Integers
- Floats
- Booleans
- Lists
- Dictionaries
- Type conversion
- Arithmetic operators
- Comparison operators
- Logical operators
- Membership operators
- Assignment operators
- if
- elif

- else
- for loops
- Indexing
- User-defined functions
- Searching
- Updating data
- Deleting data
- Basic calculations
- Data validation

Removed from the required topics

I would **remove while loops, match, slicing, and context managers (with) from the Python-topic checklist.**

They aren't core learning objectives for this particular inventory problem. with is naturally encountered through `st.spinner()`, but it doesn't need to be presented as a Python topic the student must deliberately practice.

Streamlit Topics Covered

- `st.set_page_config()`
- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`
- `st.text_input()`
- `st.number_input()`
- `st.selectbox()`
- `st.button()`
- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`
- `st.form()`

- `st.columns()`
 - `st.sidebar`
 - `st.tabs()`
 - Text formatting
 - Text alignment
 - Primary/secondary buttons
-

Learning Objectives

After completing this project, the student should be able to:

- Design a medium-sized inventory application.
 - Store related information using lists and dictionaries.
 - Validate user input before modifying data.
 - Prevent duplicate records.
 - Prevent negative inventory.
 - Search records using multiple criteria.
 - Update existing records.
 - Delete records safely.
 - Perform calculations using stored data.
 - Generate meaningful reports.
 - Organize a Streamlit application using tabs, columns, forms, and a sidebar.
 - Use Streamlit status messages appropriately.
 - Apply basic visual formatting to improve usability.
 - Build a practical application without using a console interface.
-

What I changed from your original version

The biggest improvement is that the problem now has **one clear architecture**:

Data → Add → View/Search → Stock Management → Reports

I also removed the duplicated console + GUI missions. The old **"Expected Console Output"** no longer belongs in a Streamlit-only assignment.

I also changed a few potentially confusing requirements:

- **"Exit"** was removed because a Streamlit app doesn't need a console-style exit button.
- **Search** was moved into the Inventory section instead of being treated as a separate application mode.

- **Delete, Restock, Remove Stock, and Update Price** are grouped into **Stock Management**.
- The **low-stock threshold** is explicitly defined as $\text{quantity} < \text{threshold}$.
- The **price update** bonus is now a normal feature, because it fits naturally into inventory management.
- `while True` was removed because Streamlit reruns the script in response to interaction.
- The console output was removed entirely.
- The formatting requirements are now **specific enough that the student actually practices them**, rather than simply being told to "use styling."
- **`st.set_page_config()`** is the new formatting/UI concept I added.
- The Multi-Warehouse task remains challenging, but is separated from the main project so it doesn't make the intermediate problem unnecessarily huge.